



CodeSmile

Reverse Engineering, Test Plan & Change Requests

Nome e Cognome	Matricola
Daniele Pio Scaparra	NF22500101
Grazia Bonagura	NF22500105
Antonio Nappi	In attesa

A.A. 2025/2026

Contents

1	Reverse Engineering dell'Architettura	3
1.1	Overview dei Package	3
1.2	Analisi dei Sottosistemi	4
1.3	Diagrammi UML delle Classi	5
1.3.1	Code Extractor	5
1.3.2	Data Preparation	5
1.3.3	Detection Rules	6
1.4	Focus su Componenti Critiche	6
2	Change Requests	8
2.1	CR1 — Miglioramento del Detector “In-Place APIs Misused”	8

2.2	CR2 — Creazione del Call Graph del Progetto	9
2.3	CR3 — Analisi e Visualizzazione del Call Graph	9
3	Valutazione del Rischio e dell’Impatto	11
4	Test Plan	11
4.1	Strategia generale	11
4.2	Test previsti per ciascuna CR	12
4.3	Criteri di successo	13
5	Benefici per il Progetto CodeSmile	13

1. Reverse Engineering dell'Architettura

Il primo passo essenziale per l'avvio del progetto consiste in una comprensione approfondita del funzionamento del tool; a tal fine è stata condotta un'estesa attività di reverse engineering dell'applicazione. Nell'ambito di tale analisi, i diagrammi UML sono stati generati automaticamente mediante **Pyreverse**¹, e successivamente rielaborati tramite **Graphviz**², unitamente a tecniche di rifinitura grafica realizzate direttamente in **L^AT_EX**. L'obiettivo dell'analisi è stato quello di ricostruire in modo chiaro e completo l'architettura del sistema, isolandone i principali componenti e comprendendo il flusso logico che governa l'intero processo di analisi.

L'osservazione dei diagrammi UML ha evidenziato un'architettura modulare, stratificata in sottosistemi indipendenti ma fortemente cooperanti. Ogni package svolge un ruolo specifico nel workflow: dall'estrazione delle informazioni dal codice Python, alla rilevazione degli smell, fino alla generazione dell'output finale. Nei paragrafi seguenti si offre una descrizione dettagliata dei vari componenti, accompagnata dai relativi diagrammi ottenuti tramite reverse engineering.

1.1 Overview dei Package

La Figura 1 mostra la struttura ad alto livello del progetto, organizzata in macro-package con responsabilità distinte. Il package **code_extractor** si occupa di estrarre informazioni strutturali dal codice; **detection_rules** implementa l'insieme di regole che individuano i code smell; **components** rappresenta il cuore orchestratore della pipeline; **smell_ai** contiene il supporto ai modelli LLM; infine, **report** gestisce la generazione dei risultati finali.

¹PyReverse è un componente della suite **pylint** deputato alla produzione automatica di UML (package e class diagram) a partire da codice Python; documentazione disponibile su: <https://pylint.readthedocs.io/en/latest/pyreverse.html>.

²Graphviz è una libreria open-source per la visualizzazione di grafi in formato DOT; sito ufficiale: <https://graphviz.org/>.

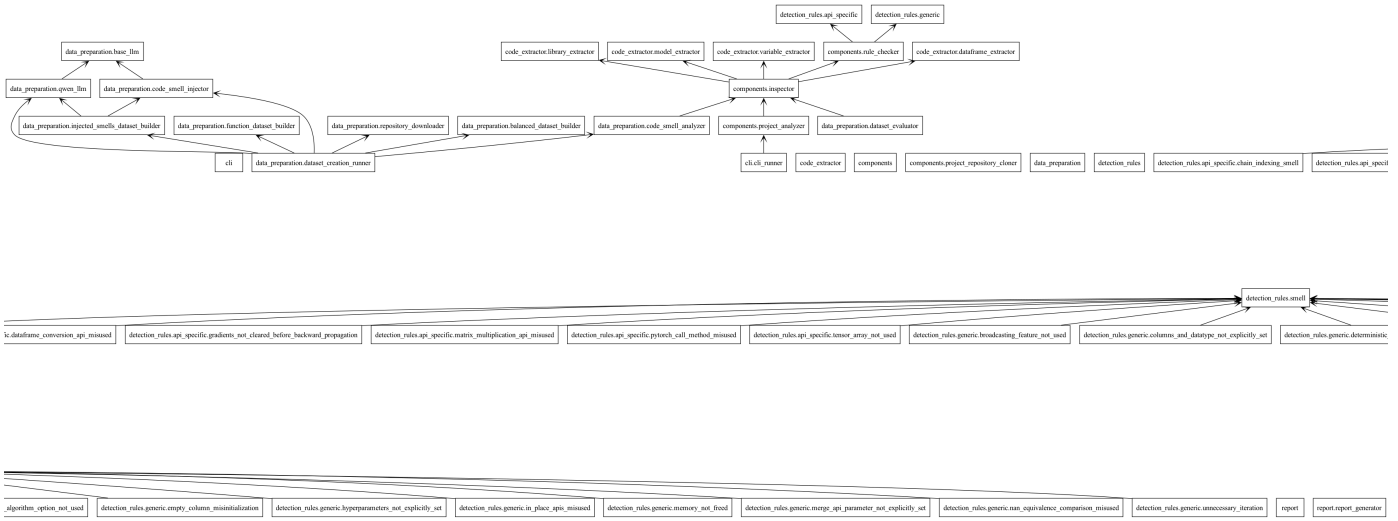


Figure 1: Overview dei macro-package del progetto CodeSmile.

Nota: il diagramma è stato suddiviso in tre segmenti verticali esclusivamente per ragioni di leggibilità e impaginazione. Il contenuto è riportato integralmente.

1.2 Analisi dei Sottosistemi

La seguente figura approfondisce due sottosistemi fondamentali: `code_extractor` e `data_preparation`. Il primo implementa diversi estrattori specializzati che navigano l'AST dei file Python per individuare funzioni, variabili, modelli, dataframe e librerie. Il secondo sottosistema raccoglie tutti i componenti dedicati alla produzione, bilanciamento e valutazione dei dataset, necessari all'addestramento e testing dei modelli LLM integrati nel progetto.

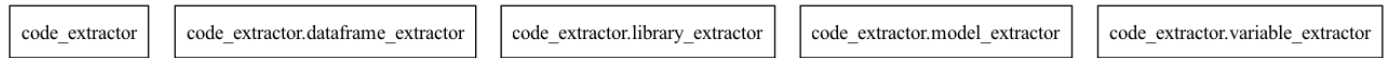
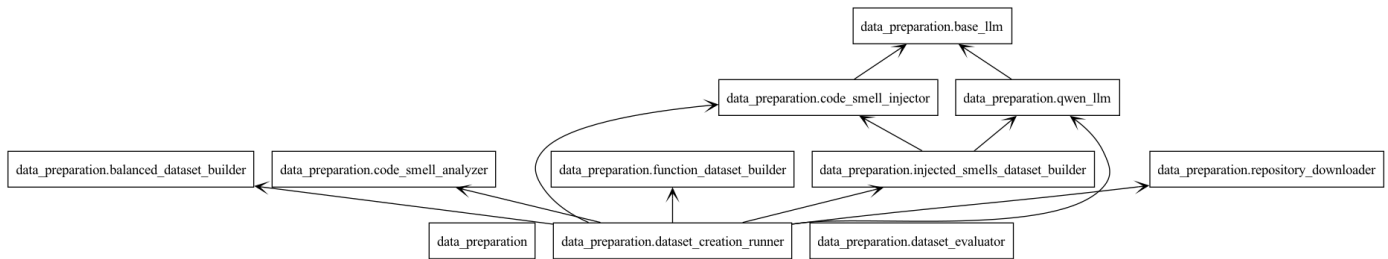
(a) Package `code_extractor`.(b) Package `data_preparation`.

Figure 2: Sottosistemi ottenuti tramite reverse engineering.

1.3 Diagrammi UML delle Classi

1.3.1 Code Extractor

Il package `code_extractor` presenta una struttura altamente modulare. Ogni estrattore è dedicato a un tipo specifico di informazione: `DataFrameExtractor` individua e analizza operazioni su dataframe; `ModelExtractor` rileva modelli e pipeline ML; `VariableExtractor` traccia le variabili definite nell'AST; `LibraryExtractor` identifica le dipendenze importate. Questa suddivisione consente un'estensione semplice e una manutenzione agevolata.

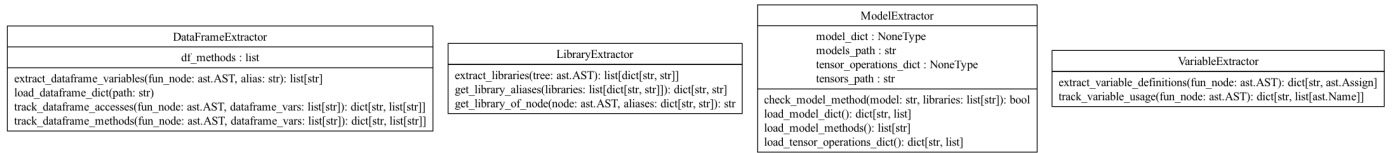


Figure 3: Class diagram del package `code_extractor`.

1.3.2 Data Preparation

Il package `data_preparation` comprende i moduli dedicati alla costruzione dei dataset utilizzati da CodeSmile e dai suoi modelli LLM. Tra le classi principali vi sono: `InjectedSmellsDatasetBuilder`, responsabile della creazione di dataset annotati con smell; `BalancedDatasetBuilder`, che genera dataset bilanciati tramite oversampling/undersampling; `QwenLLM` e `BaseLLM`, che forniscono interfacce uniformi per l'interazione con i modelli di linguaggio. Il design risulta orientato alla componibilità e alla sperimentazione.

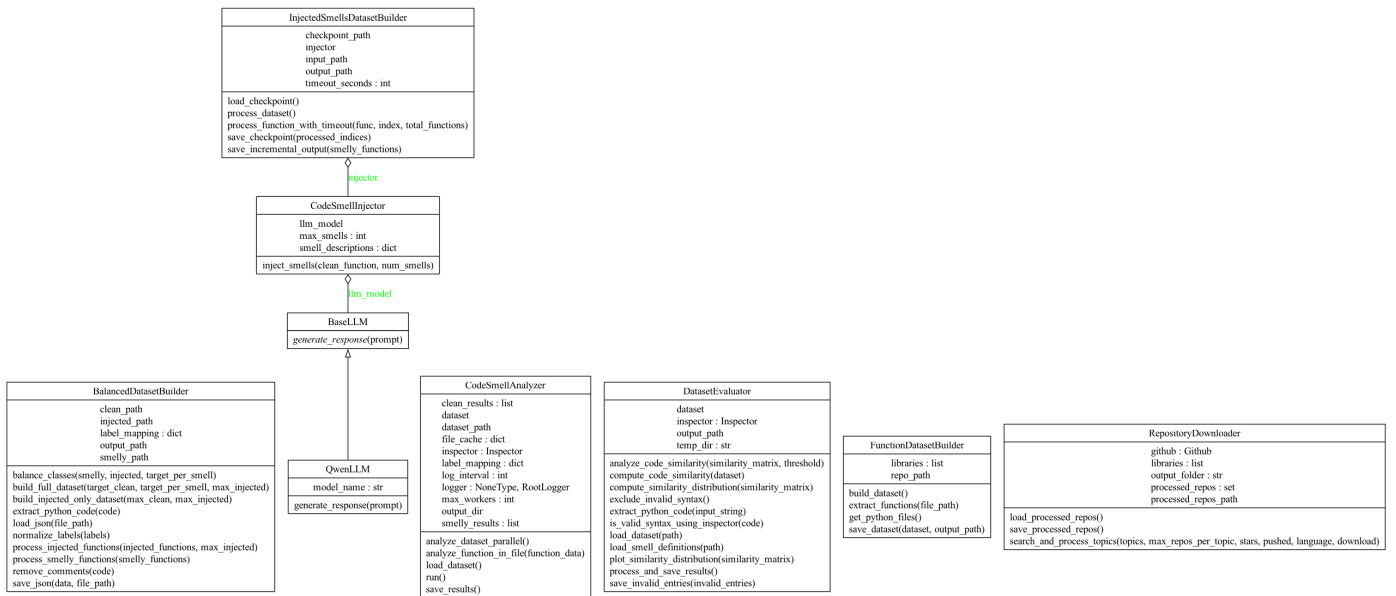


Figure 4: Class diagram del package `data_preparation`.

1.3.3 Detection Rules

Il package `detection_rules` contiene l'insieme estensibile di regole per l'identificazione dei code smell. Il diagramma mostra due livelli principali:

- smell **generici**, applicabili a codice Python indipendentemente dalla libreria utilizzata;
- smell **API-specific**, che riconoscono pattern ricorrenti in librerie come Pandas, NumPy e PyTorch.

Ogni smell è implementato come classe autonoma che eredita da una struttura comune, seguendo un approccio progettuale che facilita l'aggiunta di nuove regole.

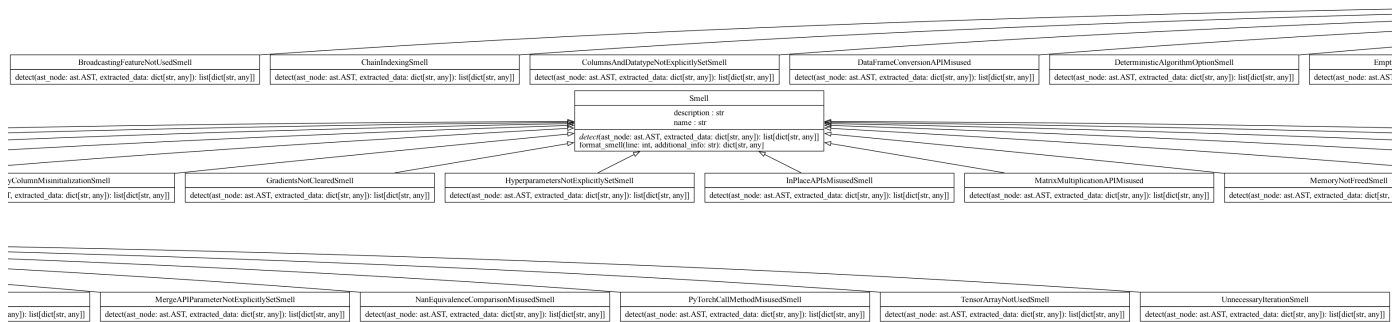


Figure 5: Class diagram del package `detection_rules`.

Nota: la segmentazione verticale consente di visualizzare correttamente un diagramma originariamente troppo esteso per la pagina.

1.4 Focus su Componenti Critiche

L'ultimo diagramma approfondisce il package `smell_ai`, responsabile della gestione dei modelli di linguaggio utilizzati per rilevare smell complessi e di natura semantica. Tra i componenti principali si evidenziano:

- classi dedicate alla generazione e personalizzazione dei prompt;
- builder per dataset sintetici e bilanciati;
- funzioni di inferenza e valutazione dei risultati prodotti dal modello.

Il package costituisce l'elemento di maggiore innovazione di CodeSmile poiché permette l'integrazione di tecniche di AI all'interno di un sistema di static analysis.



7

2. Change Requests

In seguito alla fase di reverse engineering sono state individuate tre aree di intervento che permettono di incrementare la capacità analitica del tool e la sua completezza funzionale. Le tre modifiche proposte rappresentano miglioramenti coerenti con gli obiettivi del progetto e consentono di evolvere CodeSmile sia in direzione di una migliore qualità dei detector sia verso la rappresentazione architetturale dei progetti analizzati.

2.1 CR1 — Miglioramento del Detector “In-Place APIs Misused”

Tipologia di manutenzione: correttiva e perfettiva.

Il detector attuale risulta incompleto e talvolta impreciso. In particolare, sono stati identificati limiti relativi a:

- copertura parziale delle API Pandas;
- assenza totale del supporto API NumPy;
- mancato riconoscimento dei risultati delle operazioni non utilizzati;
- presenza di falsi positivi in contesti corretti (assegnazioni, funzioni wrapper, ecc.).

Obiettivi.

- Ampliare la copertura delle API Pandas e NumPy rilevanti.
- Migliorare la capacità del detector di identificare risultati ignorati.
- Ridurre la frequenza di falsi positivi e falsi negativi.

Benefici.

- Aumento della precision e recall del detector.
- Minore rumorosità nei report finali.
- Risultati più coerenti con la semantica delle API ML e Data Science.

2.2 CR2 — Creazione del Call Graph del Progetto

Tipologia di manutenzione: evolutiva.

Attualmente CodeSmile analizza funzioni in modo isolato senza modellare relazioni o dipendenze. L'assenza di un call graph impedisce di avere una visione architeturale del progetto e limita la capacità degli utenti di comprendere il contesto in cui avvengono gli smell rilevati.

Obiettivi.

- Estrarre tutte le chiamate tra funzioni e metodi all'interno del progetto.
- Costruire automaticamente il call graph.
- Annotare ogni nodo con eventuali smell rilevati.
- Esportare il grafo nei formati JSON e DOT.
- Estendere la CLI con un'opzione dedicata alla generazione del grafo.

Benefici.

- Introduzione di un livello architeturale finora assente nel tool.
- Supporto agli utenti nella comprensione della struttura del progetto.
- Base necessaria per le analisi avanzate previste in CR3.

2.3 CR3 — Analisi e Visualizzazione del Call Graph

Tipologia di manutenzione: evolutiva.

Il grafo generato tramite CR2 necessita di un modulo dedicato alla sua analisi e visualizzazione. Un file JSON o DOT non è sufficiente per mettere in evidenza conflitti, nodi critici, cicli o funzioni centrali.

Obiettivi.

- Produrre una visualizzazione grafica leggibile del call graph (PNG/SVG).
- Calcolare metriche di centralità e dipendenze funzionali.
- Identificare automaticamente cicli tra funzioni/metodi.
- Fornire una rappresentazione chiara ed esportabile per l'utente finale.

Benefici.

- Migliore interpretabilità dell'architettura del progetto.
- Individuazione immediata di componenti critiche o complesse.
- Strumento utile per attività di refactoring e manutenzione.

Mockup della visualizzazione

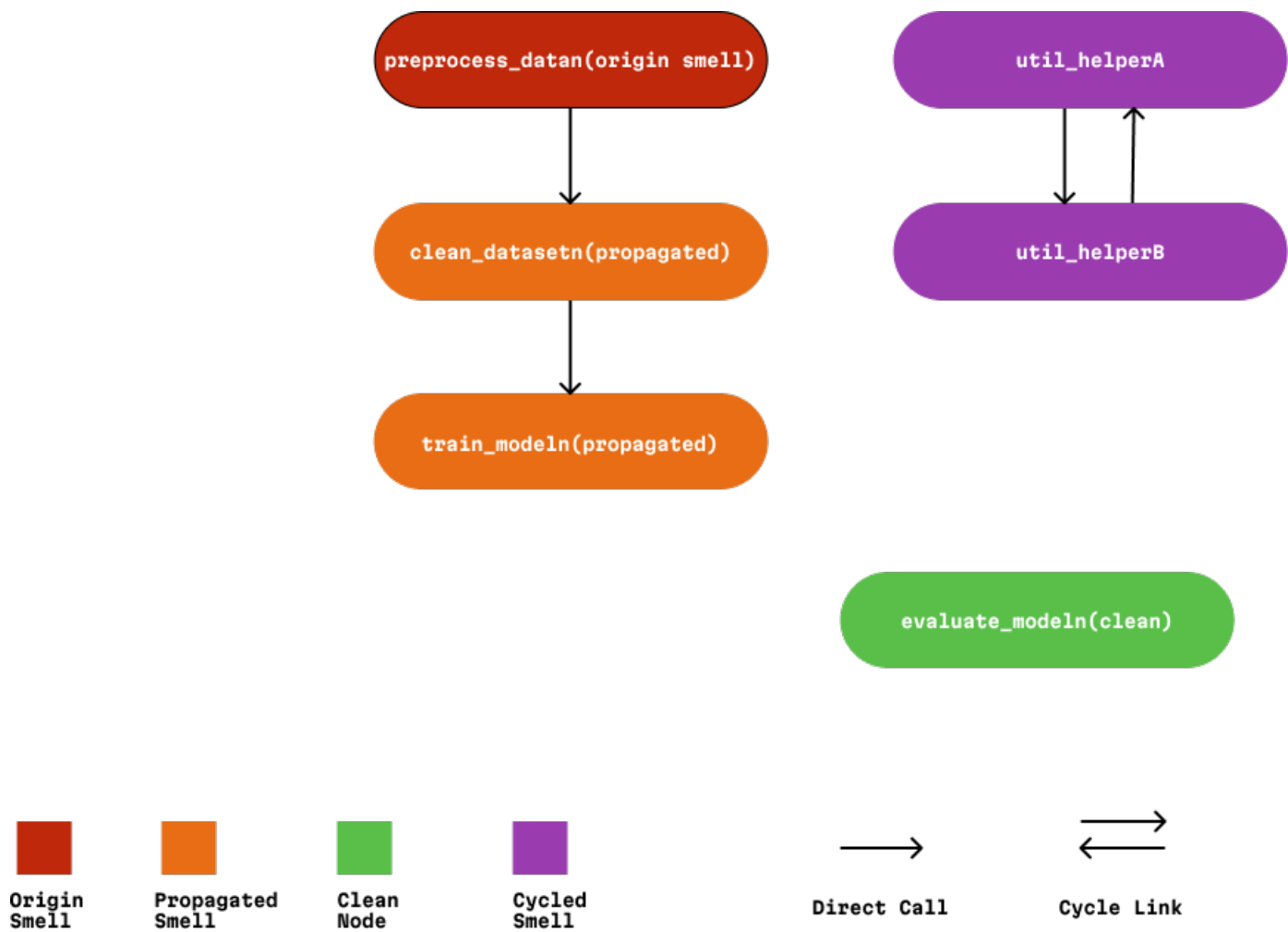


Figure 7: Mockup della visualizzazione del Call Graph prevista dalla CR3.

Nota: l'immagine mostrata rappresenta un mockup concettuale della UI prevista per la visualizzazione e l'analisi del Call Graph. Il mockup verrà sostituito con la versione definitiva realizzata in sede di implementazione.

3. Valutazione del Rischio e dell'Impatto

La seguente tabella riassume la classificazione del rischio e dell'impatto delle tre CR:

Change Request	Rischio	Impatto
CR1 — Detector IPA Misused	Medio	Medio
CR2 — Call Graph Generation	Medio	Alto
CR3 — Visualizzazione & Analisi del Call Graph	Basso-Medio	Alto

Table 1: Valutazione preliminare del rischio e dell'impatto delle CR.

4. Test Plan

Il Test Plan definisce gli approcci, le tecniche e i casi di test necessari per validare l'implementazione delle Change Request, garantendo al contempo l'assenza di regressioni rispetto al comportamento attuale del sistema.

4.1 Strategia generale

La strategia si basa su un approccio multilivello:

Unit Testing.

- Detector migliorato (CR1).
- Moduli di estrazione del call graph (CR2).
- Funzioni di analisi strutturale e metriche (CR3).

Integration Testing.

- Pipeline: estrazione $\rightarrow$ annotazione smell $\rightarrow$ call graph $\rightarrow$ analisi.
- Verifica consistenza tra CLI, motore static analysis e generatori di file.

System Testing.

- Esecuzione completa del tool su un progetto multi-file.
- Validazione della correttezza del grafo generato.

Regression Testing.

- Tutti i test già presenti nel repository.
- Esclusione dei test non rilevanti (GUI, WebApp).

4.2 Test previsti per ciascuna CR**Test CR1 — Detector IPA Misused.**

- Verifica casi corretti (API non-in-place con assegnazione).
- Verifica casi scorretti (risultati non utilizzati).
- Test dedicati a Pandas e NumPy.
- Test su riduzione dei falsi positivi.

Test CR2 — Costruzione Call Graph.

- Corretto riconoscimento delle chiamate tra funzioni.
- Corretta annotazione dei nodi con gli smell.
- Verifica JSON e DOT generati.
- Test CLI `-call-graph`.

Test CR3 — Analisi e Visualizzazione.

- Generazione corretta PNG/SVG.
- Calcolo metriche (degree, betweenness).
- Identificazione cicli.
- Coerenza visualizzazione $\leftrightarrow$ struttura effettiva.

4.3 Criteri di successo

La modifica sarà considerata accettata se:

- tutti i test CR-specific superano l'esecuzione;
- nessuna regressione è introdotta nelle componenti esistenti;
- i file generati risultano validi e interpretabili;
- l'esecuzione end-to-end del tool è stabile.

5. Benefici per il Progetto CodeSmile

Le tre CR contribuiscono a migliorare il tool su più livelli.

Benefici funzionali.

- CR1 aumenta l'accuratezza dei risultati.
- CR2 introduce una prospettiva architetturale fondamentale.
- CR3 consente interpretazioni visive e metriche avanzate.

Benefici manutentivi.

- Codice più modulare e analizzabile.
- Migliore chiarezza interna nelle componenti principali.
- Test suite più solida e completa.